📂 CATEGORY 3: Resources, Data Sources & Dependencies

> **Difficulty:** Beginner → Intermediate | **Topics:** 6 | **Terraform Interview Mastery Series**

Table of Contents

1. Resource Block Anatomy and Lifecycle
2. Data Sources — What They Are, When to Use Them vs Resources
3. ⚠ Implicit vs Explicit Dependencies
4. `depends_on` — When It's Needed and When It's Misused
5. Resource Replacement vs In-Place Update — What Triggers Each
6. ⚠ `-target` Flag — Power and Danger

Topic 12: Resource Block Anatomy and Lifecycle

🔵 What It Is (Simple Terms)

A **resource block** is the fundamental unit of Terraform configuration. It declares a piece of infrastructure you want to exist — an EC2 instance, an S3 bucket, a DNS record, a Kubernetes namespace. Everything Terraform manages is a resource.

Understanding the full anatomy of a resource block — and what happens to it across its lifecycle — is the bedrock of writing correct Terraform.

🔵 Why It Exists — What Problem It Solves

Without resource blocks, there is no Terraform. Resources are the declaration of intent — "I want this infrastructure to exist, with these attributes." Terraform Core reads these declarations and reconciles reality to match them.

🔵 Full Resource Block Anatomy

```
# Syntax:
# resource "<RESOURCE_TYPE>" "<LOCAL_NAME>" { ... }
#          └── provider_type.resource └── name in this module

resource "aws_instance" "web_server" {
  # ─────────────────────────────────────────────
  # REQUIRED ARGUMENTS — vary by resource type
  # ─────────────────────────────────────────────
```

```
  ami           = "ami-0c55b159cbfafe1f0"   # what image to use
  instance_type = "t3.medium"               # compute size

  # ──────────────────────────────────────────────────
  # OPTIONAL ARGUMENTS
  # ──────────────────────────────────────────────────
  subnet_id              = aws_subnet.private.id    # reference to another
resource
  vpc_security_group_ids = [aws_security_group.web.id]
  key_name               = var.key_pair_name
  iam_instance_profile   = aws_iam_instance_profile.web.name

  # ──────────────────────────────────────────────────
  # NESTED BLOCKS — structured sub-configurations
  # ──────────────────────────────────────────────────
  root_block_device {
    volume_type           = "gp3"
    volume_size           = 50
    delete_on_termination = true
    encrypted             = true
  }

  ebs_block_device {
    device_name = "/dev/sdf"
    volume_size = 100
    volume_type = "gp3"
    encrypted   = true
  }

  # ──────────────────────────────────────────────────
  # META-ARGUMENTS — control Terraform behavior
  # (not passed to the provider)
  # ──────────────────────────────────────────────────
  count    = 2                              # create 2 instances
  # for_each = var.instance_map           # OR use for_each (not both)

  depends_on = [aws_iam_role_policy.web]  # explicit dependency

  provider = aws.us_east_1                # use specific provider instance

  lifecycle {
    create_before_destroy = true
    prevent_destroy       = false
    ignore_changes        = [ami, tags["LastUpdated"]]
    replace_triggered_by  = [aws_launch_template.web.id]
  }

  # ──────────────────────────────────────────────────
  # PROVISIONERS — post-creation actions
  # (use sparingly — anti-pattern in most cases)
  # ──────────────────────────────────────────────────
  provisioner "remote-exec" {
    inline = ["sudo apt-get update", "sudo apt-get install -y nginx"]
    connection {
```

```
      type        = "ssh"
      user        = "ubuntu"
      private_key = file("~/.ssh/id_rsa")
      host        = self.public_ip
    }
  }

  # ──────────────────────────────────────────────
  # TAGS — metadata on the resource
  # ──────────────────────────────────────────────
  tags = {
    Name        = "web-server-${count.index}"
    Environment = var.environment
    ManagedBy   = "terraform"
  }
}
```

🔵 Resource Addressing

Every resource has a unique address used in CLI commands and cross-references:

```
Format: <resource_type>.<local_name>
        <resource_type>.<local_name>[index]        (count)
        <resource_type>.<local_name>["key"]        (for_each)

Examples:
  aws_instance.web_server
  aws_instance.web_server[0]                       (count = 2, index 0)
  aws_instance.web_server["prod"]                  (for_each with "prod"
key)
  module.vpc.aws_subnet.private                    (inside a module)
  module.vpc.module.subnet.aws_subnet.private      (nested modules)
```

🔵 Computed vs Known Attributes

Terraform distinguishes between two types of resource attributes:

```
resource "aws_instance" "web" {
  # KNOWN before apply — you provide these
  ami           = "ami-0c55b159cbfafe1f0"    # known
  instance_type = "t3.medium"                # known

  # COMPUTED after apply — AWS assigns these
  # id          = (known after apply)         # AWS assigns instance ID
  # public_ip   = (known after apply)         # AWS assigns IP
  # arn         = (known after apply)         # AWS constructs ARN
  # private_dns = (known after apply)
```

```
}

# Computed attributes appear in plan output as "(known after apply)"
# They become available in state after apply
# You can reference them: aws_instance.web.id, aws_instance.web.public_ip
```

🔵 The Resource Lifecycle — 4 Phases

```
┌──────────────────────────────────────────────────────────────────┐
│                    Resource Lifecycle                            │
│                                                                  │
│  1. PLAN PHASE                                                   │
│     Core reads .tf config                                        │
│     Provider validates config schema                             │
│     Provider calls PlanResourceChange()                          │
│     Returns: create / update / replace / no-op / delete          │
│                                                                  │
│  2. APPLY PHASE (Create)                                         │
│     Provider calls ApplyResourceChange() → Create                │
│     Real resource created in cloud                               │
│     Provider returns all attributes (including computed)         │
│     Core writes resource to state                                │
│                                                                  │
│  3. APPLY PHASE (Update / Replace)                               │
│     Provider calls ApplyResourceChange() → Update or Replace     │
│     In-place update: existing resource modified                  │
│     Replace: old resource deleted, new resource created          │
│     State updated with new attributes                            │
│                                                                  │
│  4. DESTROY PHASE                                                │
│     Provider calls ApplyResourceChange() → Delete                │
│     Real resource deleted from cloud                             │
│     Resource removed from state                                  │
└──────────────────────────────────────────────────────────────────┘
```

🔵 Resource Arguments: Required vs Optional vs Computed

Every provider documents three categories of arguments:

Category	Meaning	Example
Required	Must be set or Terraform errors	`ami`, `instance_type`
Optional	Has a default or can be omitted	`monitoring = false`
Computed	Set by the provider after creation	`id`, `arn`, `public_ip`
Optional + Computed	Can be set OR computed if not set	`availability_zone`

```
resource "aws_instance" "web" {
  ami           = "ami-0c55b159cbfafe1f0"  # Required
  instance_type = "t3.medium"             # Required
  monitoring    = true                    # Optional (default: false)
  # id is Computed — don't set it, AWS assigns it
  # availability_zone is Optional+Computed — set it or let AWS choose
}
```

🔵 Provisioners — When and Why to Avoid

```
# Provisioners run scripts on resources after creation
# ⚠ They are an anti-pattern in modern Terraform

resource "aws_instance" "web" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t3.micro"

  # ⚠ Anti-pattern: use user_data or cloud-init instead
  provisioner "remote-exec" {
    inline = ["sudo apt install nginx -y"]
  }
}

# ✅ Better: use user_data for instance initialization
resource "aws_instance" "web" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t3.micro"
  user_data     = templatefile("${path.module}/scripts/init.sh", {
    environment = var.environment
  })
}
```

Why provisioners are anti-patterns:

- They run only on creation — not idempotent
- They require network connectivity to the instance
- Failures leave resources in a broken half-created state
- They blur the line between Terraform (infra) and Ansible (config)

🔵 Short Interview Answer

> "A resource block has a type (like `aws_instance`), a local name for referencing it within the module, required and optional arguments that configure it, nested blocks for structured sub-configurations, and meta-arguments like `count`, `for_each`, `depends_on`, `lifecycle`, and `provider` that control Terraform's behavior rather than the resource itself. Resources go through a lifecycle of plan →

> create/update/replace → destroy. Computed attributes like IDs and ARNs are assigned by the provider after creation and become available for other resources to reference."

🔵 Common Interview Questions

Q: What is the difference between an argument and a meta-argument?

> "Arguments are configuration passed to the provider — they define what the resource should look like (AMI, instance type, tags). Meta-arguments are instructions to Terraform Core itself — `count` controls how many instances to create, `depends_on` adds explicit dependencies, `lifecycle` controls replacement behavior, `provider` selects which provider instance to use. Meta-arguments are not passed to the provider API — they only affect how Terraform manages the resource."

Q: What happens to state if `terraform apply` fails halfway through?

> "Terraform updates state after each individual resource operation succeeds. If apply fails halfway, the state file reflects all resources that were successfully created or modified before the failure. Resources that failed are either absent from state (if they were being created) or in their pre-apply state (if they were being modified). Re-running `terraform apply` will attempt to converge the remaining resources. This is why Terraform apply is not atomic — partial applies are a real scenario to handle."

Q: What are provisioners and why should you avoid them?

> "Provisioners run scripts on a resource after it's created — `remote-exec` SSHs in and runs commands, `local-exec` runs commands on the machine running Terraform. They're considered an anti-pattern because they only run at creation time (not idempotent), they require network access, failures leave resources in broken states, and they mix infrastructure provisioning with configuration management. Better alternatives are `user_data` for EC2 initialization, cloud-init, Packer for pre-baked AMIs, or dedicated tools like Ansible for post-provisioning config."

🔵 Gotchas & Edge Cases

- ⚠ **Resource type name comes from the provider** — `aws_instance` means the `aws` provider manages a resource type called `instance`. The first segment maps to the provider.
- **Local name must be unique within a module** — two `resource "aws_instance" "web"` blocks error. But `aws_instance.web` and `aws_ec2_fleet.web` are fine — different types.
- **`self` reference in provisioners** — inside a provisioner block, `self` refers to the enclosing resource. `self.public_ip` gets the IP of the instance being provisioned.
- **Computed attributes during plan** — if resource B references a computed attribute of resource A that doesn't exist yet, plan shows `(known after apply)` for B's attribute too. This propagates through the graph.
- **`null` values** — setting an argument to `null` tells Terraform to use the provider default for that attribute, not to set it to a string "null".

🔵 Connections to Other Concepts

- → **Topic 13 (Data Sources):** Data sources are read-only resources
- → **Topic 14 (Dependencies):** Resource references create implicit dependencies
- → **Topic 16 (Replacement):** `lifecycle` block controls replacement behavior
- → **Category 4 (Variables/Expressions):** Resource attributes are accessed via expressions
- → **Category 5 (Meta-arguments):** `count`, `for_each`, `lifecycle` are all meta-arguments

---

Topic 13: Data Sources — What They Are, When to Use Them vs Resources

---

🔵 What It Is (Simple Terms)

A **data source** is a read-only query. It fetches information about infrastructure that already exists — without creating, modifying, or destroying anything. Think of it as a `SELECT` statement against your cloud API.

```
# Resource block — creates/manages infrastructure
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"           # Terraform creates this VPC
}

# Data source block — reads existing infrastructure
data "aws_vpc" "existing" {
  id = "vpc-0a1b2c3d4e5f67890"         # Terraform reads this VPC (created
elsewhere)
}
```

🔵 Why It Exists — What Problem It Solves

Real-world infrastructure is never entirely managed by a single Terraform configuration. You often need to:

- Reference a **VPC created by another team's Terraform config**
- Look up the **latest AMI ID** without hardcoding it
- Read **account-level information** like your AWS account ID or region
- Reference resources **created manually** or by other tooling
- Look up **shared infrastructure** (central DNS zones, shared subnets, transit gateways)

Data sources bridge the gap between what Terraform manages and what already exists in the world.

🔵 Data Source Anatomy

```
# Syntax:
# data "<DATA_SOURCE_TYPE>" "<LOCAL_NAME>" { ... }
```

```
data "aws_ami" "ubuntu" {
  most_recent = true                        # get the latest matching AMI
  owners      = ["099720109477"]            # Canonical (Ubuntu publisher)

  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*"]
  }

  filter {
    name   = "virtualization-type"
    values = ["hvm"]
  }
}

# Reference the data source result
resource "aws_instance" "web" {
  ami           = data.aws_ami.ubuntu.id    # use fetched AMI ID
  instance_type = "t3.medium"
}
```

Key difference in referencing:

```
# Resources:    <resource_type>.<name>.<attribute>
aws_instance.web.id
aws_vpc.main.cidr_block

# Data sources: data.<data_source_type>.<name>.<attribute>
data.aws_ami.ubuntu.id
data.aws_vpc.existing.cidr_block
data.aws_caller_identity.current.account_id
```

🔵 Common Data Sources You Must Know

AWS Account & Identity

```
data "aws_caller_identity" "current" {}
# Returns: account_id, user_id, arn

data "aws_region" "current" {}
# Returns: name (e.g. "us-east-1"), description

data "aws_availability_zones" "available" {
  state = "available"
}
# Returns: names (list of AZ names), zone_ids

# Usage:
```

```
resource "aws_s3_bucket" "audit" {
  bucket = "audit-logs-${data.aws_caller_identity.current.account_id}"
}
```

AMI Lookup (Critical — Always Use This)

```
# Never hardcode AMI IDs — they differ by region and change with updates
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name   = "name"
    values = ["al2023-ami-*-x86_64"]
  }
}

data "aws_ami" "ubuntu_22" {
  most_recent = true
  owners      = ["099720109477"]  # Canonical

  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*"]
  }
}
```

VPC & Networking

```
# Look up a VPC by tag
data "aws_vpc" "shared" {
  tags = {
    Name        = "shared-services-vpc"
    Environment = "production"
  }
}

# Look up subnets in that VPC
data "aws_subnets" "private" {
  filter {
    name   = "vpc-id"
    values = [data.aws_vpc.shared.id]
  }
  tags = {
    Tier = "private"
  }
}

# Look up a specific subnet
```

```
data "aws_subnet" "primary" {
  vpc_id            = data.aws_vpc.shared.id
  availability_zone = "us-east-1a"
  tags              = { Tier = "private" }
}
```

IAM

```
# Look up an existing IAM policy
data "aws_iam_policy" "readonly" {
  arn = "arn:aws:iam::aws:policy/ReadOnlyAccess"
}

# Build an IAM policy document (no API call — pure HCL)
data "aws_iam_policy_document" "assume_role" {
  statement {
    effect = "Allow"
    principals {
      type        = "Service"
      identifiers = ["ec2.amazonaws.com"]
    }
    actions = ["sts:AssumeRole"]
  }
}

resource "aws_iam_role" "web" {
  assume_role_policy = data.aws_iam_policy_document.assume_role.json
}
```

Route53

```
data "aws_route53_zone" "primary" {
  name         = "example.com."
  private_zone = false
}

resource "aws_route53_record" "web" {
  zone_id = data.aws_route53_zone.primary.zone_id
  name    = "app.example.com"
  type    = "A"
  ttl     = 300
  records = [aws_instance.web.public_ip]
}
```

Terraform Remote State (Cross-Stack References)

```
# Read outputs from another Terraform state file
data "terraform_remote_state" "vpc" {
  backend = "s3"
  config = {
    bucket = "mycompany-terraform-state"
    key    = "networking/vpc/terraform.tfstate"
    region = "us-east-1"
  }
}

# Use outputs from the VPC stack in this stack
resource "aws_instance" "app" {
  subnet_id =
data.terraform_remote_state.vpc.outputs.private_subnet_ids[0]
}
```

🔵 Data Sources vs Resources — Decision Guide

```
┌──────────────────────────────────────────────────────────────┐
│            When to Use Resource vs Data Source               │
│                                                              │
│  Use a RESOURCE when:                                        │
│  ✅ You want Terraform to create and own this infrastructure │
│  ✅ The resource should be destroyed when `terraform destroy` runs │
│  ✅ You need to configure the resource's attributes          │
│  ✅ Changes to config should update the real resource        │
│                                                              │
│  Use a DATA SOURCE when:                                     │
│  ✅ The infrastructure already exists (created elsewhere)    │
│  ✅ You only need to READ attributes, not modify anything    │
│  ✅ The resource is managed by another team/stack            │
│  ✅ You want to look up dynamic values (latest AMI, account ID) │
│  ✅ You're referencing shared/central infrastructure         │
└──────────────────────────────────────────────────────────────┘
```

Scenario	Resource	Data Source
Create a new VPC	✅	
Reference a VPC created by networking team		✅
Create an EC2 instance	✅	
Find the latest Ubuntu AMI		✅
Create an IAM role	✅	
Build an IAM policy document		✅ (`aws_iam_policy_document`)
Create a Route53 hosted zone	✅	

Scenario	Resource	Data Source
Look up an existing hosted zone to add records		✅
Get the current AWS account ID		✅

🔵 When Data Sources Are Evaluated

```
terraform plan:
  → Data sources are read DURING the plan phase
  → Results are used to compute the full plan
  → If a data source query fails, the whole plan fails

Exception: data sources that depend on resources not yet created
  → Those are read during apply (after their dependencies are created)
  → This shows as "(known after apply)" in plan output for dependent
resources
```

🔵 Short Interview Answer

> "Data sources are read-only queries — they fetch information about existing infrastructure without creating or modifying anything. The canonical use cases are: looking up the latest AMI ID, reading attributes of infrastructure managed by another team's Terraform config (like a shared VPC), and getting account-level info like the current AWS account ID or region. Unlike resources (which are prefixed with the resource type), data sources are referenced with `data.<type>.<name>.<attribute>`. They're evaluated during the plan phase, before resources are created."

🔵 Real World Production Example

```
# Complete example: deploy an app onto existing shared infrastructure
# The VPC and subnets are managed by the networking team — we just read
them

# ── Read shared infrastructure ──────────────────────────────────────
data "aws_vpc" "shared" {
  tags = { Name = "production-vpc", Team = "networking" }
}

data "aws_subnets" "app_private" {
  filter {
    name   = "vpc-id"
    values = [data.aws_vpc.shared.id]
  }
  tags = { Tier = "app-private" }
}
```

```
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]
  filter {
    name   = "name"
    values = ["al2023-ami-*-x86_64"]
  }
}

data "aws_caller_identity" "current" {}
data "aws_region" "current" {}

# ── Create app infrastructure ──────────────────────────────────────
resource "aws_instance" "app" {
  count         = 3
  ami           = data.aws_ami.amazon_linux.id          # dynamic AMI
  instance_type = "t3.medium"
  subnet_id     = data.aws_subnets.app_private.ids[count.index %
length(data.aws_subnets.app_private.ids)]

  tags = {
    Name      = "app-server-${count.index}"
    AccountId = data.aws_caller_identity.current.account_id
    Region    = data.aws_region.current.name
  }
}

resource "aws_s3_bucket" "app_data" {
  bucket = "app-
data-${data.aws_caller_identity.current.account_id}-${data.aws_region.curr
ent.name}"
}
```

🔵 Common Interview Questions

Q: What is `data "aws_iam_policy_document"` and why is it special?

> "It's a data source that generates an IAM policy JSON document from HCL. Unlike most data sources, it doesn't make any API calls — it's entirely local computation. You define policy statements in HCL and it outputs the correctly formatted JSON string. This is the recommended approach for IAM policies in Terraform because it avoids embedding raw JSON strings, enables expression interpolation (like using account IDs or ARNs), and provides type checking for policy structure."

Q: What happens if a data source returns no results?

> "It depends on the data source. Some error immediately — `data.aws_vpc.main` with a filter that matches no VPCs will error with 'no matching VPC found'. Others return empty collections — `data.aws_subnets.private` might return an empty list of IDs if no subnets match. The key is that data source failures propagate to plan failures — the entire plan is aborted. Always make sure your

> data source filters are accurate, especially in environments (dev/staging/prod) where resource names or tags might differ."

Q: What is `terraform_remote_state` and when would you use it?

> "`terraform_remote_state` reads the state outputs of another Terraform configuration. It's used to share information between separate Terraform stacks — for example, a networking stack outputs VPC IDs and subnet IDs, and an application stack reads them via `terraform_remote_state`. The alternative is to use a tool like Terragrunt or read from a parameter store (SSM, Consul). The downside of `remote_state` is tight coupling between stacks and the requirement that the reader has access to the state bucket."

🔵 Gotchas & Edge Cases

- ⚠ **Data sources are re-read on every `terraform plan`** — if the underlying resource changes outside Terraform, the next plan picks up the new values. This can cause unexpected changes in dependent resources.
- ⚠ **`most_recent = true` in AMI data sources** — this can cause instance replacement if a new AMI is published and `ignore_changes = [ami]` is not set in the lifecycle block.
- **Data source vs `terraform_remote_state`** — prefer SSM Parameter Store or similar for cross-stack data sharing. `remote_state` creates a dependency on the exact state backend structure.
- **`data.aws_subnets` vs `data.aws_subnet`** — `aws_subnets` (plural) returns a list of IDs. `aws_subnet` (singular) returns a single subnet's full attributes. Use plural for lists, singular when you need specific attributes.
- **Data sources evaluated during plan can cause slow plans** — if you have 50 data source lookups, each making an API call, your plan can take minutes. Batch where possible.
- **Data sources in modules** — they still need provider access. If a module uses `data.aws_ami`, the module needs the AWS provider passed to it.

🔵 Connections to Other Concepts

- → **Topic 12 (Resources):** Data sources complement resources — read what exists, create what doesn't
- → **Topic 14 (Dependencies):** Data sources can create implicit dependencies just like resources
- → **Category 7 (Modules):** `terraform_remote_state` is the classic inter-module data sharing pattern
- → **Category 6 (State):** Remote state data source reads another config's state file

---

Topic 14: ⚠ Implicit vs Explicit Dependencies

🔵 What It Is (Simple Terms)

Terraform needs to know the **order** in which to create, update, and destroy resources. If a subnet must exist before an EC2 instance is launched into it, Terraform must create the subnet first. Dependencies define this order.

Terraform has two types:

- **Implicit dependencies** — Terraform figures them out automatically by analyzing resource references
- **Explicit dependencies** — you declare them manually using `depends_on`

> ⚠ Understanding the difference — and knowing when each is needed — is tested heavily in interviews. Most candidates know `depends_on` exists but can't explain WHY implicit dependencies work or when they're insufficient.

🔵 Why It Exists — What Problem It Solves

Infrastructure has natural ordering requirements:

```
Can't create EC2 instance before its subnet exists
Can't create subnet before its VPC exists
Can't attach an IAM policy before both the role and policy exist
Can't create an RDS instance before its security group exists
```

Terraform solves ordering by building a **Directed Acyclic Graph (DAG)** of all resources. Dependencies become directed edges in the graph. Resources with no dependencies can be created in parallel — those with dependencies wait.

🔵 Implicit Dependencies — How They Work

An implicit dependency is created automatically when one resource **references an attribute** of another.

```
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
}

resource "aws_subnet" "private" {
  vpc_id     = aws_vpc.main.id       # ← IMPLICIT DEPENDENCY created here
  cidr_block = "10.0.1.0/24"        # aws_subnet.private depends on
aws_vpc.main
}

resource "aws_instance" "web" {
  ami       = "ami-0c55b159cbfafe1f0"
  subnet_id = aws_subnet.private.id  # ← IMPLICIT DEPENDENCY
  # aws_instance.web depends on aws_subnet.private
}
```

The resulting DAG:

```
aws_vpc.main
    │
    └──► aws_subnet.private
              │
              └──► aws_instance.web
```

Terraform creates them in this order: VPC → Subnet → Instance. On destroy, the order is reversed: Instance → Subnet → VPC.

Rule: Any time you write `resource_type.resource_name.attribute` in a configuration, you create an implicit dependency on that resource.

🔵 Parallel Execution via the DAG

The real power of the DAG: **independent resources run in parallel**.

```
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
}

# These three subnets all depend on the VPC
# But they don't depend on each other — created in parallel
resource "aws_subnet" "private_a" {
  vpc_id            = aws_vpc.main.id    # depends on VPC
  cidr_block        = "10.0.1.0/24"
  availability_zone = "us-east-1a"
}

resource "aws_subnet" "private_b" {
  vpc_id            = aws_vpc.main.id    # depends on VPC
  cidr_block        = "10.0.2.0/24"
  availability_zone = "us-east-1b"
}

resource "aws_subnet" "private_c" {
  vpc_id            = aws_vpc.main.id    # depends on VPC
  cidr_block        = "10.0.3.0/24"
  availability_zone = "us-east-1c"
}
```

Execution:

```
Step 1: aws_vpc.main (must be first)
Step 2: aws_subnet.private_a  ┐
```

```
        aws_subnet.private_b  ├── All three in parallel
        aws_subnet.private_c  ┘
```

Default parallelism is 10 concurrent operations (`-parallelism=10`).

🔵 Explicit Dependencies — When Implicit Isn't Enough

Implicit dependencies only work when you reference an attribute. But sometimes a dependency exists **without an attribute reference** — typically when one resource's behavior affects another through side effects.

Classic example: IAM policy attachment

```
resource "aws_iam_role" "web" {
  name               = "web-role"
  assume_role_policy = data.aws_iam_policy_document.assume.json
}

resource "aws_iam_role_policy_attachment" "web_s3" {
  role       = aws_iam_role.web.name       # implicit dependency on role
✅
  policy_arn = "arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess"
}

resource "aws_instance" "web" {
  ami                  = data.aws_ami.ubuntu.id
  instance_type        = "t3.medium"
  iam_instance_profile = aws_iam_instance_profile.web.name   # implicit
dependency

  # ⚠ PROBLEM: The instance starts BEFORE the S3 policy is attached
  # The instance references the instance_profile (which references the
role)
  # But it does NOT reference the policy attachment
  # Terraform sees no implicit dependency between aws_instance.web
  # and aws_iam_role_policy_attachment.web_s3
  # The instance boots before it has the permissions it needs

  # ✅ FIX: explicit dependency
  depends_on = [aws_iam_role_policy_attachment.web_s3]
}
```

🔵 The Dependency Graph Visualized

```
Without explicit dependency (WRONG):
  aws_iam_role.web ──► aws_iam_role_policy_attachment.web_s3
  aws_iam_role.web ──► aws_iam_instance_profile.web ──► aws_instance.web
```

```
  (instance can start before policy is attached — race condition)

With explicit dependency (CORRECT):
  aws_iam_role.web ──► aws_iam_role_policy_attachment.web_s3
                                    │
  aws_iam_role.web ──► aws_iam_instance_profile.web ──► aws_instance.web
                                                              ▲
  aws_iam_role_policy_attachment.web_s3 ──────────────────────┘
(depends_on)
```

🔵 The `terraform graph` Command

```
# Generate the dependency graph in DOT format
terraform graph

# Visualize with graphviz
terraform graph | dot -Tpng > graph.png
terraform graph | dot -Tsvg > graph.svg

# Sample output (simplified):
digraph {
  compound = "true"
  "[root] aws_vpc.main" -> "[root]
provider[\"registry.terraform.io/hashicorp/aws\"]"
  "[root] aws_subnet.private" -> "[root] aws_vpc.main"
  "[root] aws_instance.web" -> "[root] aws_subnet.private"
}
```

🔵 Short Interview Answer

> "Terraform builds a dependency graph to determine resource creation and destruction order. Implicit dependencies are created automatically when one resource references another's attribute — like `subnet_id = aws_subnet.private.id` creates an implicit dependency on the subnet. Independent resources are created in parallel. Explicit dependencies using `depends_on` are needed when a real dependency exists but no attribute is referenced — the classic example is IAM policy attachments that need to complete before an EC2 instance starts, even though the instance doesn't reference the policy attachment directly."

🔵 Real World Production Example

```
# Real scenario: Lambda function needs S3 bucket permissions before
execution
# The Lambda is triggered by S3 events — it needs the bucket notification
# AND the permission to be set before it can receive events
```

```
resource "aws_s3_bucket" "data" {
  bucket = "myapp-data-bucket"
}

resource "aws_lambda_permission" "s3_invoke" {
  statement_id  = "AllowS3Invoke"
  action        = "lambda:InvokeFunction"
  function_name = aws_lambda_function.processor.function_name
  principal     = "s3.amazonaws.com"
  source_arn    = aws_s3_bucket.data.arn
}

resource "aws_s3_bucket_notification" "trigger" {
  bucket = aws_s3_bucket.data.id

  lambda_function {
    lambda_function_arn = aws_lambda_function.processor.arn
    events              = ["s3:ObjectCreated:*"]
  }

  # ✅ Notification config MUST wait for permission to be set first
  # Without this, S3 can't invoke Lambda when notification triggers
  depends_on = [aws_lambda_permission.s3_invoke]
}
```

🔵 Common Interview Questions

Q: How does Terraform determine the order to create resources?

> "Terraform builds a Directed Acyclic Graph (DAG) where each resource is a node and dependencies are directed edges. When resource A references an attribute of resource B, Terraform adds an edge from A to B, meaning B must be created before A. Terraform then walks this graph, creating resources in dependency order. Resources with no dependency relationship are created in parallel, up to the parallelism limit (default 10). On destroy, the graph is traversed in reverse."

Q: Can you create a circular dependency in Terraform?

> "Terraform detects circular dependencies (cycles in the DAG) and immediately errors with 'Cycle' error during plan. For example, if resource A depends on resource B and resource B depends on resource A, Terraform can't determine an order and errors out. The fix is usually to extract the dependency into a third resource, use `data` sources, or restructure the configuration. Common causes are two resources that both try to reference each other's computed attributes."

🔵 Gotchas & Edge Cases

- ⚠ **Implicit dependencies are on the resource, not the attribute** — `subnet_id = aws_subnet.private.id` makes the instance depend on the ENTIRE subnet resource being complete, not just the `id` attribute. Terraform waits for the full resource creation.

- ⚠ **Data sources can also create implicit dependencies** — `data.aws_ami.ubuntu.id` creates an implicit dependency on the `data.aws_ami.ubuntu` data source being resolved.
- **`depends_on` on a module** — you can add `depends_on` to an entire module call, which makes every resource in that module depend on the specified resources. Use carefully — it's very broad.
- **Destroy order** — dependencies are reversed on destroy. If A depends on B (B created first), then on destroy A is destroyed first, then B. This is automatic and handled by the DAG.
- **Cycles with modules** — cycles can be harder to debug when they span module boundaries. Use `terraform graph` to visualize.

🔵 Connections to Other Concepts

- → **Topic 15 (`depends_on`):** The mechanism for explicit dependencies
- → **Category 5 (Meta-arguments):** `depends_on` is a meta-argument
- → **Category 10 (Troubleshooting):** Cycle errors are a dependency graph problem
- → **Category 10 (`terraform graph`):** Visualizing the dependency graph

Topic 15: `depends_on` — When It's Needed and When It's Misused

🔵 What It Is (Simple Terms)

`depends_on` is a meta-argument that adds an **explicit, manual dependency** between resources. It tells Terraform: "don't create/modify/destroy this resource until these other resources are done."

```
resource "aws_instance" "web" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = "t3.medium"

  depends_on = [
    aws_iam_role_policy_attachment.web_s3,
    aws_security_group_rule.allow_https
  ]
}
```

🔵 Why It Exists — What Problem It Solves

Terraform's implicit dependency detection is powerful but not omniscient. It can only see dependencies expressed through **attribute references**. When a dependency exists due to **side effects** — behaviors that happen at the cloud API level — Terraform can't detect them automatically.

`depends_on` is the escape hatch for these cases.

🔵 When `depends_on` IS Needed

Case 1: IAM Propagation Delays

```
resource "aws_iam_role" "lambda" {
  name               = "lambda-execution-role"
  assume_role_policy = data.aws_iam_policy_document.lambda_assume.json
}

resource "aws_iam_role_policy_attachment" "lambda_basic" {
  role       = aws_iam_role.lambda.name
  policy_arn = "arn:aws:iam::aws:policy/service-
role/AWSLambdaBasicExecutionRole"
}

resource "aws_lambda_function" "processor" {
  filename      = "lambda.zip"
  function_name = "data-processor"
  role          = aws_iam_role.lambda.arn          # implicit dep on role
✅
  handler       = "index.handler"
  runtime       = "nodejs18.x"

  # ✅ NEEDED: Lambda can be created before policy attachment completes
  # AWS IAM is eventually consistent — the role needs its policies before
  # Lambda can actually execute. Without this, Lambda creation may succeed
  # but the first invocation fails with permission errors.
  depends_on = [aws_iam_role_policy_attachment.lambda_basic]
}
```

Case 2: Resource Behavior Side Effects

```
resource "aws_s3_bucket_policy" "public_read" {
  bucket = aws_s3_bucket.website.id
  policy = data.aws_iam_policy_document.public_read.json
}

resource "aws_cloudfront_distribution" "website" {
  # References the bucket domain — implicit dep on bucket ✅
  origin {
    domain_name = aws_s3_bucket.website.bucket_regional_domain_name
    origin_id   = "S3Origin"
  }

  # ✅ NEEDED: CloudFront setup requires bucket policy to be applied first
  # Without the policy, CloudFront can't access S3 objects during setup
  depends_on = [aws_s3_bucket_policy.public_read]
}
```

Case 3: VPC Endpoint Dependencies

```
resource "aws_vpc_endpoint" "s3" {
  vpc_id       = aws_vpc.main.id
  service_name = "com.amazonaws.us-east-1.s3"
  route_table_ids = [aws_route_table.private.id]
}

resource "aws_instance" "app" {
  ami           = data.aws_ami.amazon_linux.id
  instance_type = "t3.medium"
  subnet_id     = aws_subnet.private.id

  # ✅ NEEDED: Instance should route S3 traffic through VPC endpoint
  # The instance doesn't reference the endpoint directly (no attribute
ref)
  # But functionally, the endpoint should exist before the instance starts
  depends_on = [aws_vpc_endpoint.s3]
}
```

Case 4: `depends_on` on Modules

```
module "database" {
  source = "./modules/rds"
  # ...
}

module "application" {
  source = "./modules/ec2"

  # ✅ All resources in application module wait for entire database module
  depends_on = [module.database]
}
```

🔵 When `depends_on` is NOT Needed (Misuse)

This is the more important part — most incorrect `depends_on` usage slows Terraform down and makes configs harder to read.

```
# ❌ WRONG — depends_on is redundant, implicit dependency already exists
resource "aws_subnet" "private" {
  vpc_id     = aws_vpc.main.id       # already creates implicit dependency
  cidr_block = "10.0.1.0/24"

  depends_on = [aws_vpc.main]        # ← REDUNDANT, remove this
}
```

```
# ❌ WRONG — over-using depends_on
resource "aws_instance" "web" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = "t3.medium"
  subnet_id     = aws_subnet.private.id   # implicit dep on subnet
  vpc_security_group_ids = [aws_security_group.web.id]  # implicit dep on
SG

  depends_on = [
    aws_subnet.private,              # ← REDUNDANT
    aws_security_group.web,          # ← REDUNDANT
    aws_vpc.main                     # ← REDUNDANT (via subnet)
  ]
}
```

Why redundant `depends_on` is harmful:

```
1. Serializes operations that could run in parallel
2. Makes the dependency graph unnecessarily complex
3. Masks the real dependency structure — harder to reason about
4. Slows down apply and destroy operations
5. Creates confusion about why the dependency was added
```

🔵 The `depends_on` Impact on Replacement

```
resource "aws_instance" "web" {
  # ...
  depends_on = [aws_iam_role_policy_attachment.web]
}
```

When `depends_on` references a resource that changes:

- Terraform may **force replacement** of the dependent resource
- This is because Terraform treats `depends_on` targets as part of the resource's "configuration" for replacement decisions
- This is a known sharp edge — `depends_on` can trigger unexpected replacements

---

🔵 Decision Framework for `depends_on`

```
Do I need depends_on here?

Step 1: Is there already an attribute reference between these resources?
  YES → depends_on is REDUNDANT. Don't add it.
```

```
    NO  → Continue to Step 2

  Step 2: Would a race condition or ordering issue occur without it?
    Does resource A need resource B to be FULLY APPLIED (not just created)
    before A can function correctly?
    YES → depends_on is NEEDED
    NO  → Don't add it

  Step 3: Is this due to cloud-level eventual consistency?
    (IAM propagation, DNS propagation, policy attachments taking effect)
    YES → depends_on is NEEDED, and consider adding time_sleep too
    NO  → Reconsider if the dependency is real
```

🔵 Short Interview Answer

> "`depends_on` adds explicit dependencies when Terraform can't detect them through attribute references. The classic cases are IAM policy attachments where the policy needs to propagate before a Lambda or EC2 instance can use it, S3 bucket policies that must exist before CloudFront accesses the bucket, and VPC endpoint configuration that should be ready before instances start. The key misuse to avoid is adding `depends_on` where an implicit dependency already exists via attribute reference — this is redundant and serializes operations that could run in parallel, slowing Terraform down."

🔵 Real World Production Example

```
# Production EKS setup — ordering matters critically

resource "aws_eks_cluster" "main" {
  name     = "production-cluster"
  role_arn = aws_iam_role.eks_cluster.arn

  vpc_config {
    subnet_ids = aws_subnet.private[*].id
  }

  # Must wait for IAM role policies to be fully attached
  depends_on = [
    aws_iam_role_policy_attachment.eks_cluster_policy,
    aws_iam_role_policy_attachment.eks_vpc_resource_controller
  ]
}

resource "aws_eks_node_group" "workers" {
  cluster_name  = aws_eks_cluster.main.name
  node_role_arn = aws_iam_role.eks_nodes.arn
  subnet_ids    = aws_subnet.private[*].id

  scaling_config {
```

```
    desired_size = 3
    max_size     = 10
    min_size     = 1
  }

  # Wait for node IAM policies AND the cluster to be ready
  depends_on = [
    aws_iam_role_policy_attachment.eks_worker_node_policy,
    aws_iam_role_policy_attachment.eks_cni_policy,
    aws_iam_role_policy_attachment.ecr_read_only,
  ]
  # Note: implicit dep on aws_eks_cluster.main via cluster_name reference
}
```

🔵 Common Interview Questions

Q: What's the difference between `depends_on` and an implicit dependency?

> "An implicit dependency is automatically detected when resource A references resource B's attribute — Terraform infers B must exist before A. `depends_on` is explicit — you manually declare the dependency for situations where A depends on B at the infrastructure behavior level but doesn't reference any of B's attributes directly. Implicit is preferred because it accurately represents the real dependency. `depends_on` is an escape hatch for side-effect-based dependencies that can't be expressed through attribute references."

Q: Can `depends_on` cause unexpected resource replacements?

> "Yes — this is a known gotcha. When the target of `depends_on` changes, Terraform may treat the dependent resource's configuration as changed too, potentially triggering a replacement. This is because Terraform treats `depends_on` references as part of the resource's 'identity' for change detection purposes. It's one reason to use `depends_on` sparingly — only add it when genuinely needed, and test what happens when the dependency changes."

Q: Can you use `depends_on` on a data source?

> "Yes. Data sources support `depends_on` too. This is useful when a data source needs to read information that a resource creates. For example, if you create an EKS cluster and then use a data source to look up its OIDC provider URL, you'd add `depends_on = [aws_eks_cluster.main]` to the data source to ensure it's read after the cluster is created."

🔵 Gotchas & Edge Cases

- ⚠ **`depends_on` with `for_each`** — when the dependency target uses `for_each`, you must reference the entire resource, not individual instances: `depends_on = [aws_iam_role_policy_attachment.this]` not individual keys.
- ⚠ **`depends_on` on module outputs** — if you want a resource to depend on a module, use `depends_on = [module.name]`, not `depends_on = [module.name.some_output]` — you can't depend on outputs.

- **`depends_on` doesn't help with eventual consistency timing** — it ensures ordering but not waiting. If IAM takes 10 seconds to propagate after `terraform apply` creates the policy attachment, the downstream resource still might get permission errors. Some teams add `time_sleep` resources as workarounds.
- **Remove `depends_on` when refactoring** — when adding attribute references later, check if existing `depends_on` entries became redundant.

🔵 Connections to Other Concepts

- → **Topic 14 (Dependencies):** `depends_on` is the mechanism for explicit deps
- → **Category 5 (Meta-arguments):** `depends_on` is a meta-argument supported on all resources and modules
- → **Category 10 (Troubleshooting):** Unexpected replacements from `depends_on` is a real debugging scenario

---

Topic 16: Resource Replacement vs In-Place Update — What Triggers Each

---

🔵 What It Is (Simple Terms)

When you change a resource's configuration, Terraform has two options:

- **In-place update** — modify the existing resource without destroying and recreating it (like `aws ec2 modify-instance-attribute`)
- **Replacement** — destroy the existing resource and create a new one (shows as `-/+` in plan output)

Understanding what triggers each is critical for avoiding unplanned downtime and data loss in production.

🔵 Why It Matters

In-place updates are safe — the resource ID stays the same, no downtime.

Replacement means:

- The old resource is **deleted** (with all its data)
- A new resource is **created** (with a new ID, new IP, etc.)
- Resources that depended on the old resource's attributes may **cascade replacements**
- For stateful resources (RDS, EBS), this means **data loss unless you have backups**

🔵 What Determines In-Place vs Replacement

The provider decides — it's documented in the registry for each resource attribute:

```
Every attribute in a provider resource has one of these change behaviors:
  • Can update in-place      → changing it modifies the resource (no
replacement)
  • Forces new resource      → changing it requires destroy + recreate
  • Computed                 → set by provider, you can't change it
directly
```

🔵 Practical Examples

In-Place Updates (Safe — no replacement)

```
# Changing these attributes updates the resource in-place:

resource "aws_instance" "web" {
  # ✅ Can change without replacement:
  instance_type = "t3.large"          # Stop + resize + start (in-place)
  # tags                              # In-place
  # user_data (with no reboot)        # In-place (no reboot by default)
  # security_group_ids                # In-place
  # iam_instance_profile              # In-place
  # ebs_optimized                     # In-place (requires stop/start)
}

resource "aws_security_group" "web" {
  # ✅ Can change without replacement:
  # description — wait, actually NO (see gotchas below)
  ingress { ... }                     # In-place (rules added/removed)
  egress  { ... }                     # In-place
  tags = { ... }                      # In-place
}
```

Replacement Triggers (⚠ Causes destroy + recreate)

```
resource "aws_instance" "web" {
  # ❌ Changing these FORCES REPLACEMENT:
  ami           = "ami-newversion"    # Forces new resource
  subnet_id     = "subnet-different" # Forces new resource
  # availability_zone                # Forces new resource
  # key_name                         # Forces new resource
  # root_block_device volume_size ↓  # Forces new resource
}

resource "aws_db_instance" "main" {
  # ❌ Changing these FORCES REPLACEMENT:
  # engine                           # Forces new resource
  # engine_version (major)           # Forces new resource
  # allocated_storage (in some cases)
```

```
  # db_subnet_group_name             # Forces new resource
  # identifier                       # Forces new resource
  # ⚠ RDS replacement = DATA LOSS if no snapshot
}

resource "aws_security_group" "web" {
  # ❌ This FORCES REPLACEMENT (common gotcha):
  description = "New description"    # Forces new resource
  # vpc_id                           # Forces new resource
}
```

🔵 Reading Replacement in Plan Output

```
# In-place update:
~ resource "aws_instance" "web" {
    id            = "i-0a1b2c3d4e5f67890"  # ID unchanged
  ~ instance_type = "t3.medium" -> "t3.large"
  }

# Replacement:
-/+ resource "aws_instance" "web" {
  ~ id            = "i-0a1b2c3d4e5f67890" -> (known after apply)  # New ID
  ~ ami           = "ami-old" -> "ami-new"  # forces replacement
  }

Plan: 1 to add, 0 to change, 1 to destroy.
# NOTE: Add happens BEFORE destroy (unless create_before_destroy = false)
```

🔵 The `lifecycle` Block — Controlling Replacement

The `lifecycle` block lets you modify Terraform's default replacement behavior:

`create_before_destroy`

```
resource "aws_instance" "web" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = "t3.medium"

  lifecycle {
    create_before_destroy = true
    # DEFAULT behavior: destroy old → create new (downtime)
    # WITH this flag:   create new → destroy old (zero-downtime
replacement)
  }
}
```

```
Default (create_before_destroy = false):
  1. Destroy old resource (i-old)    ← downtime here
  2. Create new resource (i-new)

With create_before_destroy = true:
  1. Create new resource (i-new)     ← both exist momentarily
  2. Destroy old resource (i-old)    ← zero downtime
```

> ⚠ **Constraint:** `create_before_destroy` propagates through dependencies. If resource A must be replaced and has `create_before_destroy = true`, all resources that depend on A must also support being created before the old A is destroyed. This can cause conflicts.

prevent_destroy

```
resource "aws_db_instance" "production" {
  identifier = "prod-database"
  # ...

  lifecycle {
    prevent_destroy = true
    # Terraform will ERROR if anything tries to destroy this resource
    # Protects against accidental `terraform destroy` on production
databases
  }
}

# Error you'll see:
# Error: Instance cannot be destroyed
# Resource aws_db_instance.production has lifecycle.prevent_destroy
# set, but the plan calls for this resource to be destroyed.
```

ignore_changes

```
resource "aws_instance" "web" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = "t3.medium"
  user_data     = file("scripts/init.sh")

  lifecycle {
    ignore_changes = [
      ami,            # Don't replace instance when new AMI is available
      tags["LastDeployed"],  # Ignore specific tag key
      user_data,      # Don't replace if user_data changes (already
running)
    ]
    # OR: ignore_changes = all  # Ignore ALL changes after creation (use
sparingly)
```

```
  }
}
```

`replace_triggered_by` (Terraform 1.2+)

```
resource "aws_launch_template" "web" {
  name_prefix   = "web-lt-"
  image_id      = data.aws_ami.ubuntu.id
  instance_type = "t3.medium"
  user_data     = base64encode(file("scripts/init.sh"))
}

resource "aws_autoscaling_group" "web" {
  desired_capacity = 3
  min_size         = 1
  max_size         = 10

  launch_template {
    id      = aws_launch_template.web.id
    version = "$Latest"
  }

  lifecycle {
    # Force ASG replacement (rolling update) whenever launch template
changes
    replace_triggered_by = [
      aws_launch_template.web.id     # Trigger when launch template ID
changes
    ]
  }
}
```

🔵 Common Replacement Gotchas Table

Resource	Attribute	Behavior	Impact
`aws_instance`	`ami`	Forces replacement	Loss of ephemeral data
`aws_instance`	`instance_type`	In-place update	Stop/start required
`aws_instance`	`subnet_id`	Forces replacement	New private IP
`aws_security_group`	`description`	Forces replacement	Downtime if attached
`aws_security_group`	`ingress` rules	In-place	Safe

Resource	Attribute	Behavior	Impact
`aws_db_instance`	`engine_version` (major)	Forces replacement	**DATA LOSS risk**
`aws_db_instance`	`instance_class`	In-place	Brief unavailability
`aws_s3_bucket`	`bucket` name	Forces replacement	**Data stays in old bucket**
`aws_route53_record`	`name`	Forces replacement	DNS gap
`aws_iam_role`	`name`	Forces replacement	IAM propagation needed

🔵 Short Interview Answer

> "When you change a resource's configuration, Terraform either updates it in-place or destroys and recreates it — shown as `-/+` in the plan. Which happens depends on the attribute — providers document whether changing a specific attribute 'forces a new resource'. Examples that force replacement on EC2: changing the AMI, subnet, or availability zone. Safe in-place changes: instance type, security groups, tags. The `lifecycle` block lets you modify this behavior — `create_before_destroy` ensures the new resource is created before the old is destroyed (zero-downtime), `prevent_destroy` guards against accidental deletion, and `ignore_changes` tells Terraform to ignore drift on specific attributes."

🔵 Common Interview Questions

Q: You changed the `description` of an AWS security group. What happens?

> "This forces replacement — AWS doesn't allow in-place description changes. Terraform destroys the old security group and creates a new one. If any resources (EC2 instances, RDS) are currently attached to that security group, they'll lose network access during the replacement window unless you use `create_before_destroy = true` on the security group. This is a very common production gotcha — always check `terraform plan` before applying security group description changes."

Q: What is `create_before_destroy` and when should you use it?

> "By default, Terraform destroys the old resource before creating the new one during replacement — this causes downtime. `create_before_destroy = true` reverses this: creates the new resource first, then destroys the old one. Both exist simultaneously for a brief period. Use it for any resource where downtime is unacceptable — load balancers, security groups attached to running instances, DNS records. The caveat is it propagates through the dependency graph — dependent resources must also be able to coexist with two versions of the upstream resource."

Q: How would you prevent Terraform from accidentally destroying a production database?

> "Two layers: First, `lifecycle { prevent_destroy = true }` on the `aws_db_instance` resource — this makes Terraform error if any plan would destroy the resource. Second, use `deletion_protection = true` on the RDS resource itself — this tells AWS to refuse deletion API calls even if `prevent_destroy` is bypassed. You can also use `final_snapshot_identifier` to ensure a snapshot is taken before any deletion. For extra safety, use IAM policies to prevent the Terraform execution role from calling `DeleteDBInstance` in production."

🔵 Gotchas & Edge Cases

- ⚠ **`ignore_changes = all`** — means Terraform never updates the resource after creation. Config drift is completely ignored. Use only when external systems legitimately modify the resource (like ASG-managed instances).
- ⚠ **Cascading replacements** — if resource A is replaced and resource B depends on A's ID, B's configuration changes (new ID) and may also need replacement. Check plan output carefully for cascade effects.
- ⚠ **`create_before_destroy` constraint propagation** — if A has `create_before_destroy` and B depends on A, B must also have `create_before_destroy` OR Terraform errors with "cycle" because creating new A requires destroying old A which requires destroying B which creates B before new A exists.
- **`prevent_destroy` only works in Terraform** — it has no effect on manual AWS console deletions. Combine with AWS-level deletion protection.
- **`ignore_changes` with `count`/`for_each`** — ignored attributes still show in plan but with a note that changes are ignored. The plan still runs, just doesn't apply those changes.

🔵 Connections to Other Concepts

- → **Topic 12 (Resource Anatomy):** `lifecycle` block is part of resource anatomy
- → **Category 5 (Meta-arguments):** `lifecycle` is a meta-argument
- → **Category 6 (State):** Replacement creates a new state entry (new ID) — important for state management
- → **Category 8 (Security):** `prevent_destroy` is a safety control for production resources

---

Topic 17: ⚠ `-target` Flag — Power and Danger

🔵 What It Is (Simple Terms)

The `-target` flag tells Terraform to only plan/apply changes to a **specific resource or module**, ignoring all others. Instead of Terraform touching everything in your configuration, it scopes the operation to exactly what you specify.

```
# Only plan/apply this specific resource
terraform plan -target=aws_instance.web
```

```
terraform apply -target=aws_instance.web

# Target a module
terraform apply -target=module.vpc

# Target a specific instance (count or for_each)
terraform apply -target='aws_instance.web[0]'
terraform apply -target='aws_instance.web["prod"]'

# Target multiple resources
terraform apply -target=aws_instance.web -target=aws_security_group.web
```

> ⚠ This is one of the most dangerous Terraform features. Interviewers test whether you know WHEN to use it and — more importantly — when NOT to.

🔵 Why It Exists — What Problem It Solves

In a large infrastructure codebase, sometimes you need to:

- **Bootstrap a dependency** that's blocking the rest of the config
- **Recover from a partial failure** where only some resources need to be reapplied
- **Test a single resource** change in a large config without touching everything
- **Emergency production fix** on a single resource without running full apply

Without `-target`, Terraform always operates on the entire configuration, which may be too slow, too risky, or impossible when parts of the config are in an error state.

🔵 How `-target` Works Internally

```
terraform apply -target=aws_instance.web

1. Terraform builds the full dependency graph as normal
2. Identifies aws_instance.web as the target node
3. Includes ONLY:
   - The targeted resource itself
   - All direct and transitive DEPENDENCIES of the target
     (resources the target depends on)
4. EXCLUDES:
   - Resources that depend ON the target (downstream)
   - All unrelated resources
5. Applies changes only to the included set
6. Updates state only for the included resources
7. Prints a WARNING about partial applies
```

Key insight: targeting `aws_instance.web` also applies its dependencies — the subnet, security group, VPC it depends on will be included if they need changes.

🔵 Visualizing What Gets Included

```
Full dependency graph:
  aws_vpc.main
      │
      ├──► aws_subnet.private ──► aws_instance.web ──► aws_eip.web
      │                                  │
      └──► aws_security_group.web ───────┘

terraform apply -target=aws_instance.web

Included (in target + dependencies):
  ✅ aws_vpc.main (dependency of subnet)
  ✅ aws_subnet.private (dependency of instance)
  ✅ aws_security_group.web (dependency of instance)
  ✅ aws_instance.web (the target itself)

Excluded (downstream of target):
  ❌ aws_eip.web (depends ON the instance — excluded)
```

🔵 Legitimate Use Cases

Use Case 1: Bootstrapping — Chicken-and-Egg Problems

```
# Problem: You need to create an ECR repository before you can push
# the Docker image that other resources reference

# Step 1: Create just the ECR repo
terraform apply -target=aws_ecr_repository.app

# Step 2: Push your Docker image (external step)
docker push 123456789.dkr.ecr.us-east-1.amazonaws.com/app:latest

# Step 3: Apply everything else (uses the image that now exists)
terraform apply
```

Use Case 2: Emergency Production Fix

```
# Production is down — security group rule is wrong
# Full apply would touch 50 resources and take 10 minutes
# Emergency: just fix the security group

terraform apply -target=aws_security_group_rule.allow_https
# Targeted apply: 30 seconds
# Full apply: 10 minutes — too slow for an emergency
```

Use Case 3: Recovery from Partial Failure

```
# Apply failed halfway — some resources created, some didn't
# Re-running full apply works but you want to verify specific resources

terraform apply -target=aws_db_instance.main
terraform apply -target=aws_db_subnet_group.main
```

Use Case 4: Iterative Development

```
# Developing a new module — testing just the new resources
# without touching the rest of the existing infra

terraform plan -target=module.new_feature
terraform apply -target=module.new_feature
```

🔵 Why `-target` is Dangerous

Danger 1: State Drift

```
Full config has:
  aws_instance.web + aws_eip.web (elastic IP attached to instance)

You run: terraform apply -target=aws_instance.web
  → Instance is replaced (new ID)
  → aws_eip.web is excluded from the apply
  → State now has aws_eip.web pointing to the OLD instance ID
  → Next terraform plan shows aws_eip.web needs to be updated
  → But anyone reading the config thinks everything is correct
```

Danger 2: Outputs Not Updated

```
terraform apply -target=aws_instance.web
# Instance is updated but outputs referencing instance.public_ip
# are NOT recalculated in state
# Other stacks reading remote_state outputs get stale data
```

Danger 3: Masks Real Problems

```
# Config has a bug in resource B that errors on plan
# Instead of fixing B, engineer uses -target to always skip it
# Problem is hidden until an emergency
```

```
terraform apply -target=aws_instance.web    # skip the broken resource
# ← This becomes a habit — the real problem never gets fixed
```

Danger 4: The Warning Terraform Gives You

```
Warning: Resource targeting is in effect

You have specified resource targeting with the -target option.
Terraform has applied the specified resource(s) and any resources they
depended on. The resource state is now considered incomplete.

# ← Terraform itself warns you that state is now incomplete
```

🔵 The Right Way to Think About `-target`

```
┌──────────────────────────────────────────────────────────────────┐
│                -target Decision Framework                        │
│                                                                  │
│  Is this an EMERGENCY in production?                             │
│    YES → Use -target, but immediately follow with full apply     │
│    NO  → Continue                                                │
│                                                                  │
│  Is this a one-time BOOTSTRAP problem?                           │
│    YES → Use -target, document why, then apply full config       │
│    NO  → Continue                                                │
│                                                                  │
│  Is this for DEVELOPMENT/TESTING a new resource?                 │
│    YES → Acceptable, but make sure to full apply before merging  │
│    NO  → Continue                                                │
│                                                                  │
│  Any other reason?                                               │
│    → Do NOT use -target. Fix the underlying problem instead.     │
│    → Refactor configs, fix errors, use workspaces, split state   │
└──────────────────────────────────────────────────────────────────┘
```

🔵 Alternatives to `-target`

Situation	Instead of -target, use:
Only some resources changed	Trust Terraform — it only changes what needs to change
Config has errors in unrelated resources	Fix the error or use `moved` block to restructure
Want to test new module	Use a separate workspace or state

Situation	Instead of -target, use:
Slow plans due to large config	Split into smaller state files (see Category 6)
Specific resource is always erroring	Fix the root cause, don't target around it

---

🔵 Short Interview Answer

> "`-target` scopes a plan or apply to a specific resource and its dependencies, excluding everything else. Legitimate uses are bootstrapping (creating a dependency that must exist before the rest can be planned), emergency production fixes where a full apply would be too slow, and recovering from partial failures. It's dangerous because it creates state inconsistency — downstream resources are excluded from the apply but still reference the changed resources in state. The next full apply will fix this, but in the interim state is incomplete. Terraform itself warns you about this. The rule is: always follow a targeted apply with a full apply as soon as possible."

🔵 Real World Production Example

```
# Real scenario: EKS cluster bootstrap
# You need the EKS cluster to exist before the kubernetes provider
# can be configured, but the kubernetes resources are in the same config

# Step 1: Create just the infrastructure layer
terraform apply \
  -target=aws_eks_cluster.main \
  -target=aws_eks_node_group.workers \
  -target=aws_iam_role.eks_cluster \
  -target=aws_iam_role.eks_nodes

# Step 2: Now the kubernetes provider can connect to the cluster
# Apply the full config (kubernetes resources, helm charts, etc.)
terraform apply

# This is a legitimate bootstrap use case — the kubernetes provider
# can't configure itself until the EKS cluster exists
# Using -target for the initial bootstrap, then full apply is correct
```

🔵 Common Interview Questions

Q: When would you use `terraform apply -target`?

> "Three legitimate scenarios: First, bootstrapping — when a chicken-and-egg situation exists, like needing to create an ECR repository before the Docker image exists that other resources reference. Second, emergency production fixes — when production is down and you need to fix a specific resource in seconds, not wait for a full 10-minute apply. Third, recovery — after a partial failure where specific resources need reapplication. In all cases, I'd follow up with a full `terraform apply` as soon as possible to resolve the state inconsistency."

Q: Why is `-target` considered dangerous?

> "Because it creates incomplete state. When you target resource A, resources that depend on A are excluded from the apply. If A changes in a way that affects those downstream resources (new ID, new IP, new ARN), the downstream resources in state still reference the old values. This creates drift between state and reality that persists until the next full apply. In the worst case, it can cause cascading failures — the downstream resource tries to use the old reference and fails. Terraform even warns you that 'the resource state is now considered incomplete' after a targeted apply."

Q: What's the difference between `-target` and `terraform state rm`?

> "`-target` scopes what gets created/updated/destroyed during an apply — it's about execution scope. `terraform state rm` removes a resource from state entirely without touching the real infrastructure — it's about state manipulation. `state rm` is used when you want Terraform to 'forget' a resource (typically to then re-import it or because it was deleted manually). They solve different problems and are both sharp tools that require care."

🔵 Gotchas & Edge Cases

- ⚠ **`-target` with `count`/`for_each` needs quotes** — `terraform apply -target='aws_instance.web[0]'` (single quotes prevent shell from interpreting brackets)
- ⚠ **`-target` on a module includes ALL resources in the module** — you can't easily target a single resource inside a module without its full address (`module.vpc.aws_subnet.private`)
- **`-target` during destroy** — `terraform destroy -target=aws_instance.web` destroys only the targeted resource and things that depend ON it (reverse of apply — downstream, not upstream)
- **`-target` doesn't skip plan validation** — the full config is still parsed and validated. If there are HCL syntax errors elsewhere, the targeted apply still fails.
- **Never use `-target` in CI/CD pipelines** — it should be a human emergency tool only. Automated pipelines should always run full applies.
- **`-target` with `-destroy`** — combining these is very dangerous. Always double-check what you're targeting before a targeted destroy.

🔵 Connections to Other Concepts

- → **Topic 14 (Dependencies):** `-target` includes dependencies automatically
- → **Category 6 (State):** State becomes incomplete after targeted applies
- → **Category 6 (`terraform state` commands):** Alternative tools for surgical state operations
- → **Category 10 (Troubleshooting):** `-target` is a troubleshooting tool — understand its limits

---

📊 Category 3 Summary — Quick Reference Card

Topic	One-Line Summary	Interview Weight

Topic	One-Line Summary	Interview Weight
12. Resource Anatomy	Type + name + args + meta-args + lifecycle	⭐⭐⭐⭐
13. Data Sources	Read-only queries — AMIs, existing VPCs, account IDs	⭐⭐⭐⭐
14. Dependencies ⚠	Implicit via references, explicit via depends_on, DAG parallel execution	⭐⭐⭐⭐⭐
15. `depends_on`	Use for side-effect deps only — misuse serializes parallel ops	⭐⭐⭐⭐⭐
16. Replacement	`-/+` in plan, `lifecycle` block, `create_before_destroy`	⭐⭐⭐⭐⭐
17. `-target` ⚠	Emergency/bootstrap tool only — creates state inconsistency	⭐⭐⭐⭐

🔑 Category 3 — Critical Decision Trees

Should I use `depends_on`?

```
Does resource A reference any attribute of resource B?
  YES → Implicit dependency exists. depends_on is REDUNDANT.
  NO  → Does a real ordering requirement exist (side effects, IAM
propagation)?
          YES → Use depends_on
          NO  → Don't add it
```

Will this change cause replacement?

```
Check the provider docs for the attribute you're changing.
Look for "forces new resource" annotation.
Or: run terraform plan and look for -/+ in output.
If replacement is unintended:
  → Use lifecycle { ignore_changes = [attribute] }
  → Or restructure to avoid the change
  → Or use create_before_destroy to minimize impact
```

Should I use `-target`?

```
Is production down RIGHT NOW?    → Yes: use it, full apply ASAP after
Is this a bootstrap scenario?    → Yes: use it, full apply after
Any other reason?                → No: fix the root cause
```

🎯 Category 3 — Top 5 Interview Questions to Master

1. **"What's the difference between a resource and a data source?"** — resource creates/owns, data source reads existing
2. **"How does Terraform determine the order to create resources?"** — DAG, implicit deps via attribute refs, parallel for independent resources
3. **"When would you use `depends_on` and what are the risks?"** — side-effect deps, IAM propagation, not for attribute-referenced deps
4. **"You changed a security group description. What happens?"** — forces replacement (`-/+`), use `create_before_destroy` to avoid downtime
5. **"When is it acceptable to use `-target`?"** — emergency/bootstrap only, always follow with full apply, danger of state inconsistency

---

> **Next:** Category 4 — Variables, Outputs, Locals & Expressions (Topics 18–28) Type `Category 4` to continue, `quiz me` to be tested on Category 3, or `deeper` on any specific topic.